

LORDS & KINGS ENTERPRISES

WhatsApp Bot System Backup & Architecture Reference Guide

Lords & Kings Enterprises - WhatsApp Bot System Guide & Architecture Reference

Project Name: Lords & Kings Stock Management & Delivery Challan WhatsApp Bot

Repository: <https://github.com/msd20a7-rgb/whatsapp-bot>

Deployment Platform: Render Cloud (<https://whatsapp-bot-etdf.onrender.com>)(<https://whatsapp-bot-etdf.onrender.com>)

Database: Supabase PostgreSQL (`stock\_items` table)

Official Gateway: Twilio WhatsApp Business API (Twiml Engine)

Executive Summary

This system is an automated, enterprise-grade WhatsApp Bot designed for Lords & Kings Enterprises. It handles live stock tracking, dispatch log parsing, automated inventory deduction in Supabase, and dynamic PDF document generation (Live Stock Summary & Delivery Challans).

The architecture uses a Dual-Provider Model:

1. Official Twilio Gateway (Production/1-on-1): 100% compliant with Meta WhatsApp Business Policies, zero risk of account ban, powered by Twiml XML responses.
2. Unofficial whatsapp-web.js Gateway (Fallback/Groups): Chromium protocol emulation used for WhatsApp group message parsing when needed.

System Architecture


```
# Operating Mode: 'twilio' (production 1-on-1) | 'webjs' (groups) | 'hybrid' (both)
BOT_MODE=twilio

# Server Webhook Port & Host URL
PORT=3000
SERVER_BASE_URL=https://whatsapp-bot-etdf.onrender.com

# Twilio Official Credentials
TWILIO_ACCOUNT_SID=your_twilio_account_sid_here
TWILIO_AUTH_TOKEN=your_twilio_auth_token_here
TWILIO_WHATSAPP_NUMBER=whatsapp:+17372212163

# Supabase Credentials
SUPABASE_URL=https://ibflwpfzhqudjautjpaq.supabase.co
SUPABASE_KEY=your_supabase_anon_key_here
```

Database Schema (`create\_tables.sql`)

Run this SQL script inside your Supabase SQL Editor to create the required stock_items table:

```
CREATE TABLE IF NOT EXISTS stock_items (
  id BIGSERIAL PRIMARY KEY,
  party_name TEXT NOT NULL DEFAULT 'LORDS & KINGS ENTERPRISES',
  grn_number TEXT NOT NULL,
  grn_date DATE NOT NULL,
  product_name TEXT NOT NULL,
  brand TEXT NOT NULL,
  variety TEXT NOT NULL,
  lot_no TEXT NOT NULL,
  rate_type TEXT NOT NULL DEFAULT 'Bags',
  uom TEXT NOT NULL DEFAULT 'KGS',
  sub_uom TEXT,
  closing_qty NUMERIC(10, 2) NOT NULL DEFAULT 0.00,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT TIMEZONE('utc', NOW())
);

-- Index for high-performance GRN queries
CREATE INDEX IF NOT EXISTS idx_stock_items_grn ON stock_items (grn_number);
```

Key Workflows & Features

1. Automatic Dispatch Listener & Delivery Challan (DC) PDF Generation

When any user or group sends a dispatch text message (e.g.):

```
Gr1341 nz gala bigcrunch
110XF-10box

Shekar transport
Ak shiva
```

Bot Execution Steps:

1. parser.js extracts:
 - `grnNumber`: `GR1341`
 - `variety`: `nz gala bigcrunch`
 - `size`: `110XF`
 - `qtyBoxes`: `10`
 - `transportName`: `Shekar transport`
 - `partyName`: `Ak shiva`
2. findBestMatchingStockItem() searches stock_items using weighted scoring (GRN digits + Brand match + Size match).

- 3. Decrements closing_qty in Supabase (39 boxes 29 boxes).
- 4. Generates a custom styled Delivery Challan PDF using PDFKit.
- 5. Delivers the Delivery Challan PDF back to the sender instantly in WhatsApp.

2. Interactive Menu (`LAK/-0026`)

- Replying LAK/-0026 triggers the interactive menu:
- Reply `1`: Manual delivery creation with available product list & quantity validation.
 - Reply `2`: Generates and delivers a live Stock Summary PDF report formatted to official Lords & Kings layout standards.

Step-by-Step Guide to Recreate This Project From Scratch

If you ever need to set up a brand new instance or duplicate this project:

Step 1: Initialize Repository & Dependencies

```
mkdir whatsapp-bot
cd whatsapp-bot
npm init -y
npm install express twilio @supabase/supabase-js pdfkit node-cron dotenv cors whatsapp-web.js qrcode-terminal
```

Step 2: Create Files

- 1. Create services/messaging.js (Express Webhook & TwiML response builder).
- 2. Create parser.js (Regex dispatch message parser).
- 3. Create index.js (Core unified engine and PDF generators).
- 4. Create .env file with credentials.

Step 3: Configure Twilio

- 1. Sign up on [Twilio](https://www.twilio.com/).
- 2. Copy Account SID & Auth Token to .env.
- 3. Open WhatsApp Sandbox Settings in Twilio Console.
- 4. Set "WHEN A MESSAGE COMES IN" Webhook URL to: https://<your-domain>/webhook/twilio (HTTP POST).

Step 4: Local Testing via SSH Tunnel

```
# Start local bot
node index.js

# Expose local port 3000 via SSH (in a second terminal)
ssh -R 80:localhost:3000 nokey@localhost.run
```

Step 5: Production Cloud Deployment on Render

- 1. Push your code to GitHub:

```
git add .
git commit -m "Initial commit"
git push origin main
```

- 2. Create a new Web Service on [Render.com](https://render.com/).
- 3. Connect your GitHub repository (msd20a7-rgb/whatsapp-bot).
- 4. Set Environment Variables in Render: BOT_MODE=twilio, PORT=3000, SERVER_BASE_URL=https://<your-render-app>.onrender.com, TWILIO_*, SUPABASE_*.
- 5. Update Twilio Webhook URL to: https://<your-render-app>.onrender.com/webhook/twilio.

Troubleshooting & Technical Learnings

Issue	Cause	Solution
Error 21654 (ContentSid Required)	Twilio trial REST API requires pre-approved Meta template IDs for outbound REST calls.	Solved by returning native TwiML XML responses (<Response><Message>...</Message></Response>)
Windows Device Guard ngrok.exe blocked	Local security policies block binary executable downloads.	Use built-in Windows Webhook REST server. 80:localhost:3000 nokey@localhost.run) or deploy to Render.
Cloud container crash (Status 1)	Express was listening on localhost instead of network interface 0.0.0.0.	Explicitly bound Express host to 0.0.0.0 in services/messaging.js: this.expressApp.listen(this.port, '0.0.0.0', ...).

